Take-Home Project: Computer-Use Automation System

interface.ai — Engineering Team

Format: Design + working implementation + short write-up. Public GitHub repo.

Time box: No deadline. We expect a focused effort, not a polished product — see Section 5.

What we're evaluating: How you *design* and *implement* a computer-use system under realistic constraints. We care more about clear thinking, sound trade-offs, and a working core than about breadth of features.

New to some of the terminology? There's a short glossary at the end. The difficulty here is in the decisions, not the vocabulary.

1. Context

Our company builds **AI agents for banks and credit unions**. This project is about the **backend integration layer that gives those agents hands** — the system that lets an AI agent actually *operate* an institution's back-office applications to get real work done.

When a system exposes an **API, we integrate through the API** — that's always the preferred path and is out of scope here. The hard reality at banks and credit unions is the **long tail of legacy applications that have no API at all**: core banking screens, servicing tools, and admin consoles where the *only* way in is to drive the UI the way a human operator would. **This system exists for that case.** It uses an LLM ("computer use") to figure out how to accomplish a task the first time, then turns what it learned into **deterministic, replayable automation** that no longer needs the model in the loop. Each such recording becomes a reusable, reviewable, parameterized **capability** that the AI agents can invoke on demand — reliably and cheaply — without re-reasoning about the UI every time.

Framed differently: the agent-facing product decides *what* to do; **this system is how it reliably and safely does it** inside legacy bank software that offers no other way in.

interface.ai

The real environment

The applications we automate are **back-office business systems used by US banks and credit unions** — the internal apps their staff use to service accounts, process transactions, and administer the institution. Three properties of this environment shape everything:

- **Stable UIs, but real runtime errors.** These are enterprise business apps, not fast-moving consumer sites: the UI for a given app is **fairly consistent and changes slowly**, which is exactly what makes a record-once / replay-many model viable. The hard part is not constant drift — it's that a replay must **accommodate the errors and exceptional states that legitimately occur at runtime**: validation errors, "record not found," permission denials, unexpected confirmation dialogs, session/timeout expiry, transient slowness, and outright app errors. A capability that only works on the happy path is not useful in production.
- **Heterogeneous, often legacy surfaces.** A given app might be a modern web app, a *legacy* web app (server-rendered, framesets, deeply nested tables, non-semantic markup, no test IDs), or a **native desktop application**. You cannot assume a clean DOM, stable selectors, or an API. In many cases the only reliable surface is what a human operator sees and does.
- **Multi-tenant at scale.** There are **hundreds of tenants (institutions), each running ~20 apps** — thousands of app instances overall. Critically, many tenants run the *same underlying vendor product* configured, branded, and versioned differently. Automation that works for one tenant should ideally generalize — or degrade gracefully — across the others rather than being rebuilt from scratch each time.

This project asks you to build a small but real version of that idea end to end, **designed with those realities in mind** even though you'll only implement against one concrete surface.

Note: This is intentionally under-specified. Part of what we're assessing is the judgment you apply when a problem is open-ended. Where the brief doesn't dictate an answer, **make a decision, and tell us why.**

2. The problem

Build a system that can:

1. **Take a goal in natural language** for a target application (e.g. *"look up member 12345 and read their current savings balance"*, *"open a new sub-account for this member and reach the confirmation screen"*, or — if you use a public proxy target — *"add a specific item to the cart and reach the checkout review page"*).

2. **Use an LLM to accomplish that goal** by driving a real application surface — observing the current state, deciding what to do, and acting. The surface may be a browser, but treat that as one case of a more general "computer use" problem (accessibility tree, screenshot + coordinates, OS-level automation, etc. are all fair game).
3. **Record the successful run as a structured, reusable artifact** — a typed, versioned description of the flow (the steps taken, how each target element/control is identified, and any data to extract) that is decoupled from the raw model transcript.
4. **Replay that artifact deterministically** — re-run the recorded flow *without the LLM in the decision loop*, using stable element/control targeting, and report success/failure.
5. **Escalate to a human when stuck** — when the system can't safely proceed, route an intervention request to a human operator and let them **take control of the live session**, then hand control back.
6. **Stay within safety guardrails** throughout — respect an allowlist of what the agent is permitted to do, and avoid leaking or persisting sensitive data (this is regulated financial data).

The through-line to keep in mind:

The model discovers. The artifact becomes a reusable capability. Deterministic replay is how the AI agent invokes it in production.

3. Core requirements (must-have)

These are the requirements we will evaluate against. *How* you satisfy them is up to you.

3.1 Goal-driven agent loop

- Accept a goal + a target (app/URL/entry point) as input.
- Run an LLM-driven observe → decide → act loop against a live surface until the goal is met or a stopping condition is hit (max steps, timeout, dead-end).
- The agent must actually interact with a real UI (click, type, navigate, read state). You choose the mechanism (DOM-level automation, accessibility tree, screenshot + coordinates, OS-level automation, etc.). Bias toward an approach that would still work when the surface has *no* clean DOM — that's the common case in our environment.

3.2 Structured artifact (an agent-invocable capability)

After a successful run, emit a **typed, serializable artifact** that captures the reusable flow. Think of it as a **capability an AI agent can call** — so it needs a clear contract, not just a step list. At minimum it should express:

- the ordered steps / actions,

- how each target element/control is identified (with your reasoning about robustness),
- **typed input parameters** (the inputs the agent supplies per invocation, e.g. a member ID),
- **typed outputs / data to extract** and their shape (what the agent gets back),
- a checkpoint or success condition.

The artifact should be **versioned** and **reviewable** — both a human reviewer and a calling agent should be able to understand what the capability does, what it needs, and what it returns. Design the schema deliberately; it's a focal point of the evaluation.

3.3 Deterministic replay (the production execution path)

- Given a saved artifact and a set of input parameters, replay it **without invoking the LLM for decisions**. This is the path an AI agent would trigger in production.
- Replay must use stable element/control targeting, verify the checkpoint/success condition, and return any declared outputs to the caller.
- **Handle errors and exceptional states explicitly.** Because the UI is stable, the interesting failures aren't layout drift — they're runtime conditions: a validation error, a "record not found" result, a permission denial, an unexpected dialog, a session timeout, or a slow/failed load. Your replay should *detect* these and respond deliberately rather than blindly proceeding. Distinguish, in your result contract, between:
  - **expected business outcomes** the caller needs to know about (e.g. "no such member" is a legitimate result, not a crash),
  - **recoverable conditions** (e.g. dismiss a known interstitial, wait/retry a transient load), and
  - **hard failures** that should stop and surface a clear, debuggable error.
- Report a clear, structured result: success (with outputs), a known business outcome, or a failure with enough detail to debug (what step, what was expected, what was observed).

3.4 Safety & policy guardrails

- Enforce an explicit, configurable **allowlist** (e.g. permitted domains/routes, and which action types are allowed). The agent must not act outside it.
- Distinguish "safe/reversible" actions from risky/irreversible ones, and handle the risky class conservatively (block, require confirmation, or flag — your call, justify it).
- **Never persist secrets or raw sensitive data** (credentials, tokens, full PII) into artifacts or logs. Redact appropriately.

3.5 Evidence / observability

Produce enough evidence to understand and debug a run: a structured log of what the agent did and why, and at least one richer signal on failure (screenshot, DOM snapshot, trace, etc. — your choice).

3.6 Human-in-the-loop escalation & handoff

Sometimes the system can't safely finish on its own — the agent is stuck during discovery, a replay hits a condition it can't recover from, or a risky/irreversible step needs a person to decide. In those cases the system must be able to bring a human into the loop:

- **Detect and route.** Identify a stuck/blocked state and raise an **intervention request** to a human operator, carrying enough context to act on it (which capability/goal, the current step, the current state or screenshot, and *why* it stopped).
- **Take control of the live session.** Let the human **operate the same live session** the automation was using — not a fresh one — perform the manual steps, and then **hand control back** so the run can resume or complete. Preserve context and evidence across the handoff, and record what the human did.
- Think about the seam this implies: automation must be able to **pause, cede control, and resume** on the *same* session, and there must be a way to know who is (or should be) in control.

Scope note: A full real-time co-browsing operator console is out of scope. A minimal but real handoff — pause automation, expose the live session for manual control (even a bare/mock operator surface), signal resume, and capture the human's actions — plus a clear design for the rest, is what we're after. Mock the operator UI if needed, but make the **handoff mechanism and the control-transfer model** real and well-reasoned.

3.7 Design for heterogeneity & scale (design, not necessarily build)

You will implement against **one** concrete surface, but your design should have a credible answer to the environment described in Section 1. In your write-up, address:

- **Surface abstraction:** how your artifact schema and replay engine would extend from your chosen surface (e.g. a web app) to a *legacy* web app and/or a *desktop* app. What's the seam between "how we perceive/act on a surface" and "the recorded flow"?
- **Multi-tenant reuse:** hundreds of tenants run ~20 apps each, and many share the same underlying vendor product. How would you represent an artifact so it can be reused (or safely specialized/overridden) across tenants running the same app, rather than re-recorded per tenant? How do you detect and manage per-tenant/version drift?

We don't expect you to implement multi-tenant or desktop support. We *do* expect the core abstractions not to paint you into a corner.

4. Explicitly your call

We are **not** prescribing any of the following. Choose what you think is best and defend it in your write-up:

- **Language, runtime, and frameworks.**
- **LLM provider / model**, and how you prompt and structure the agent loop.
- **Computer-use technology** (Playwright, Puppeteer, Selenium, a CUA/agent SDK, screenshot-based control, accessibility APIs, OS automation frameworks, etc.).
- **Target application (a stand-in for the real thing).** We are **not** giving you access to a real bank system, and you should not try to obtain one. Pick a proxy target that lets you exercise the interesting problems: a non-trivial multi-step flow (search → detail → action, or a multi-field form with a confirmation step). Good options include a public demo/sandbox site, a local sample app you build or mock, or — if you want to lean into the "legacy/no-clean-DOM" reality — an intentionally hostile surface (iframes/framesets, table-based layouts, no test IDs) or even a simple desktop app. If you use a public site, respect its terms and rate limits, and never use real credentials or real PII.
- **The artifact schema** and how it's stored/serialized.
- **How determinism is achieved** on replay (locator strategy, fallbacks, waiting, etc.).
- **Architecture and boundaries** (single process vs. services, sync vs. queued, etc.). Simpler is fine if justified.

One thing that isn't your call: the discovery run has to be real. At least one genuine LLM-driven run against a live surface, with the evidence in `/evidence/` to show it happened. That's the heart of the project and we can't assess a description of it. You'll need your own model API access; a single successful run is not an expensive thing to produce.

Everywhere else, a clean seam is fine. The operator console, a desktop surface, anything Section 3 already lets you stub — mock it deliberately and document what you mocked and why. We'd rather see a well-designed seam than a stalled project.

5. Scope & expectations

We assume AI-assisted development. Use Cursor, Claude, Copilot, or whatever you normally reach for — we expect it and encourage it. That assumption sets the bar: with modern tooling, the scaffolding-heavy parts (agent loop, schemas, replay executor, guardrails, logging) come

together fast, so what we're looking for is a **complete end-to-end vertical slice that touches every core requirement** in Section 3 — not just one or two of them.

Concretely, we're looking for a working thread that runs all the way through:

a goal → an LLM-driven run that completes it → a saved capability artifact → a **deterministic replay** with input params, outputs, and error/outcome handling → a **human-escalation path** that can take over the live session → evidence for both runs.

Because implementation throughput is no longer the bottleneck, the real test is **judgment and integration**: the quality of your artifact schema, your locator/control-robustness strategy, your error taxonomy, your control-transfer model, and how coherently the pieces fit together. That's where we'll focus, and you should be ready to defend every decision.

Depth is where you exercise judgment, not breadth:

- **Go deep where it matters** — the artifact schema, deterministic replay plus error handling, and the safety/escalation model are the load-bearing pieces.
- **Cut depth, not whole capabilities.** Prefer a thin-but-real version of every core requirement over a polished subset. It's fine to keep something minimal, stubbed at a clean seam, or mocked (the operator UI, the desktop surface) — as long as it's intentional, documented, and the *seam and design* are real.
- **Say what you cut and why**, and what you'd build next with more time.

6. Deliverables

Please use these exact paths and headings — we read a lot of submissions side by side.

1. Source code in a public git repository, with `/README.md` covering:

- how to set up and run it (include any keys/config needed, and how to run without live services if applicable),
- a demo path: the exact command(s) to run the agent on a goal, then replay the resulting artifact.

2. A design write-up at `/REPORT.md` (~1–3 pages), using these seven headings:

1. **Architecture** — your architecture and the key decisions plus trade-offs.
2. **Artifact schema** — the schema and why you shaped it that way.
3. **Determinism & error handling** — how you make replay deterministic, and how you detect and handle runtime errors and exceptional states (and, secondarily, any UI drift).

4. **Heterogeneity & multi-tenant** — how your design extends to legacy web and desktop surfaces, and to reuse across institutions running the same app (see 3.7).
5. **Escalation & handoff** — how you detect "stuck," how a human takes control of the live session, and how control is handed back.
6. **Safety** — your guardrail model and its limits.
7. **Cuts** — what you deliberately left out, and what you'd build next.

3. A demonstration of the end-to-end flow in `/evidence/` — a saved example artifact plus logs from both a discovery run and a replay run. Ideally include **one replay that hits an error or exceptional state** (a bad input, a not-found result, or an injected/simulated failure) to show how your system detects and reports it. A short screen recording is welcome but optional.

7. Evaluation criteria

We'll weigh these roughly in this order.

System design. Clear boundaries, sensible data models, good trade-offs, appropriate simplicity. The artifact schema and replay contract are central.

Correctness of the core loop. The agent actually completes a real goal; the artifact replays deterministically and verifies success.

Robustness & error handling. How your replay detects and responds to runtime errors and exceptional states; how cleanly it separates expected business outcomes from recoverable conditions and hard failures; sound locator, wait, and checkpoint strategy.

Human-in-the-loop escalation. A real, well-reasoned mechanism to detect "stuck," route an intervention request with context, transfer control of the *live* session to a human, and resume afterward — not just a TODO.

Generalization to the real environment. A credible design story for heterogeneous surfaces and for reusing artifacts across many tenants running the same app, without brittle assumptions or per-tenant rebuilds.

Safety & data handling. Allowlist enforcement, treatment of risky and irreversible actions, redaction of regulated financial data.

Code quality. Readable, reasonably typed and tested where it counts, easy to run.

Communication. The write-up makes your reasoning, trade-offs, and cut lines clear.

We do **not** reward feature breadth, framework name-dropping, or building scaling *infrastructure* (queues, clusters, multi-tenant plumbing). Designing your core abstractions so they *could* scale to the real environment is valuable; prematurely building that infrastructure is not. A small, correct, well-argued system is the goal.

8. Optional stretch goals

Only if you have time and a solid core. Pick at most one or two — depth over breadth.

- **Agent-facing capability interface:** expose saved artifacts as a catalog of callable capabilities (a small tool/function-calling surface, or an API endpoint) that an AI agent could discover and invoke by name with typed args — and show one being invoked.
- **Code generation:** emit a runnable test or automation snippet (a page object, a test file) from an artifact.
- **Confidence & approval:** score artifacts by how reliably they replay, and gate unattended replay on an approval state (draft → approved).
- **Assisted fallback:** on replay failure, allow a bounded, policy-checked LLM recovery for a single step (never open-ended), and record it as evidence.
- **Canonicalization / cross-tenant reuse:** normalize concrete routes and values into parameterized patterns (`/item/12345` → `/item/:id`), and/or demonstrate one artifact recorded on a "base" app being applied to a second, slightly different variant — a stand-in for two tenants running the same vendor product — with per-variant overrides.
- **Multi-run stability:** replay N times and report a stability/flakiness signal.

9. Ground rules

- **AI-assisted development is assumed and encouraged.** Use whatever coding assistants, models, libraries, and tools you normally would. We care about the result, the design, and your reasoning — not whether you hand-wrote every line. The flip side: you own everything you submit and must be able to explain and defend any part of it in detail.
- Don't automate against sites where doing so would violate their terms, harm the service, or require real credentials you shouldn't use. Prefer sandboxes, demo sites, or a local app for anything sensitive.
- Keep secrets out of the repo.
- Time-box it yourself. There's no deadline, but this is not meant to consume your month — we're evaluating judgment, not endurance. If you stop early, document the rest as next steps.

10. Glossary

Not knowing these is fine. Not looking them up is the problem.

Computer use — an LLM operating a computer interface the way a person would, reading the screen or page and then clicking and typing, rather than calling an API.

DOM — the browser's structured representation of a page. A "clean DOM" has meaningful elements and stable identifiers; legacy apps often don't.

Accessibility tree — the parallel representation browsers and operating systems expose for screen readers. Often more stable than raw markup, and available on desktop apps too.

Locator / selector — how you tell automation *which* control to act on. The choice determines whether replay still works next month.

Test ID — an attribute developers add specifically so automation can find an element reliably. Legacy enterprise apps essentially never have them.

Deterministic replay — re-running a recorded flow the same way every time, with no model deciding anything. Same inputs, same steps, same outputs.

Checkpoint — a condition you assert to confirm you actually reached the state you expected, rather than assuming the click worked.

Business outcome vs. failure — "no such member" is a legitimate answer the caller needs, not a crash. Conflating the two is the most common design mistake here.

Tenant — one customer institution. Hundreds of them, many running the same vendor software configured differently.

11. Submission

Push to a **public** GitHub repo and email the link to **assignments@interface.ai**.

Put the repo URL on its own line, use the address you applied with, and don't send a zip.